



Bag of Tricks for Training-Free Vision-Language-Action Models: What to See, When to Act, and How Much to Compute?

Anonymous Authors

Abstract—Vision-Language-Action (VLA) models have shown strong performance in robotic manipulation, but their high inference cost can limit practical deployment. Recent training-free methods aim to improve efficiency without modifying or retraining the underlying policy. However, existing studies evaluate these methods on only a small number of models or benchmarks, so their behaviour across different VLA systems has not been studied in a consistent way. In this work, we present a systematic study of training-free VLA efficiency methods across three parts of the control loop: *what* the policy observes, *when* it is evaluated, and *how much* computation is used at each evaluation. Importantly, we study five representative methods (tricks) – visual foveation, action repetition, depth pruning, guarded action reuse, and temporal feature fusion. Then, we evaluate these tricks under a common protocol across seven VLA backbones and multiple manipulation environments, measuring both task performance and efficiency. Our results reveal a diverse set of trade-offs: some methods substantially reduce inference cost but degrade task performance, whereas others preserve or improve performance while providing only modest efficiency gains. These trade-offs vary considerably across VLA backbones, manipulation tasks, and evaluation settings. We hope this study provides a useful empirical basis for the design and evaluation of future efficient VLA policies.

I. INTRODUCTION

Vision-language-action (VLA) models are emerging as general-purpose policies for instruction-conditioned robotic manipulation [1]–[3]. However, their growing capabilities come with substantial inference cost. Large vision encoders and language backbones repeatedly process high-dimensional observations, while some policies additionally rely on iterative action-generation modules. In closed-loop manipulation, this cost directly affects how frequently the policy can observe the environment, update its decisions, and respond to changes during execution. Improving VLA efficiency without modifying or retraining the underlying policy is therefore important for practical deployment.

Recent training-free approaches reduce inference cost by exploiting redundancy at different stages of the VLA pipeline [4], [5]. Visual-token methods remove or reuse redundant image features [6]–[8]; depth-pruning methods skip transformer layers whose representations change little [9], [10]; and temporal methods reuse computation or actions across nearby control steps [7], [11], [12]. These methods often demonstrate favorable trade-offs between reduced computation and preserved task performance. However, such gains are typically reported for a specific trick, backbone, benchmark, and implementation [13], [14]. This leaves an important question for efficient VLA deployment:

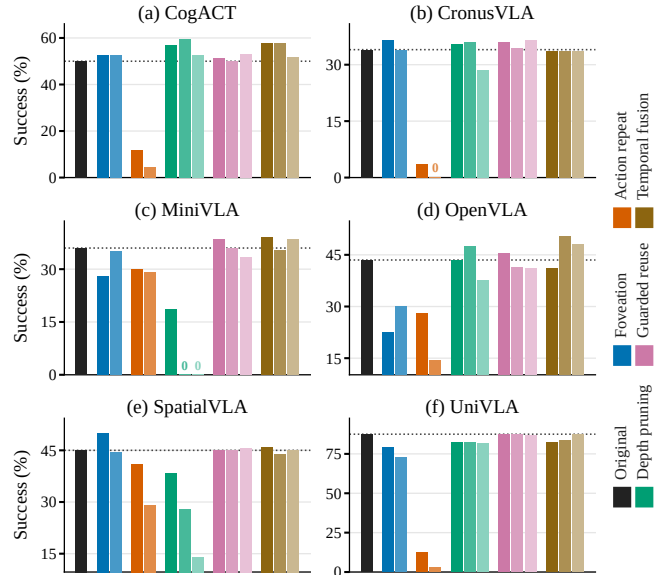


Fig. 1. Success rates across 6 VLA backbones and 14 configurations in SimplerEnv WidowX. Some tricks improve success for particular backbones, but none provides a consistent gain across models. Bars follow the setting order in Sec. IV-A from darker to lighter; dotted lines mark the original policies, and 0 denotes no successful episodes.

do training-free efficiency tricks provide consistent benefits across different VLA models and manipulation settings, or are their gains inherently configuration-dependent?

This question is particularly important for VLAs because efficiency tricks operate within a closed-loop control system. Unlike compression of a static recognition model, modifying VLA inference can change both the computational cost and the behavior of the policy. For example, action repetition reduces the number of policy evaluations but also reduces the frequency of environmental feedback. Visual foveation changes the information presented to the policy without necessarily reducing the number of visual tokens. Temporal reuse introduces information from previous observations, while depth pruning may affect different functions across different VLA architectures. Consequently, reduced computation does not necessarily lead to lower runtime or preserved manipulation performance. The effectiveness of a trick can therefore depend on the underlying architecture, action representation, and implementation.

This paper investigates this question via a systematic study of training-free efficiency *tricks* [15], [16] of VLA models. We examine when and where commonly used tricks remain effective across different backbones and manipulation envi-

ronments. We organize these tricks via three complementary dimensions of the VLA control loop: *what* visual information the policy observes, *when* the policy is evaluated, and *how much* computation is performed during each evaluation. Specifically, we evaluate visual foveation, action repetition, depth pruning, guarded action reuse, and temporal feature fusion across *seven* VLA backbones and *three* benchmark settings. All tricks are compared against the corresponding unmodified policy under matched initial conditions, identical task instances, and a common latency protocol.

Therefore, our main contributions and findings include the following: **1) Evaluation of Key Efficiency Tricks:** We systematically evaluate five representative training-free efficiency tricks – visual foveation, action repetition, depth pruning, guarded action reuse, and temporal feature fusion – that target different sources of inference cost in VLA systems. Key insights include: no single trick consistently improves both efficiency and manipulation performance across all settings (Fig. 1); the effectiveness of visual and temporal tricks varies substantially across VLA backbones and tasks; and reducing computation does not necessarily translate into proportional runtime gains or preserved task success. **2) Cross-Model and Implementation Analysis:** We further study how the behavior of these tricks changes across model architectures and implementation choices. Our empirical results show that the same trick can improve one backbone while degrading another under comparable evaluation conditions. **3) Comprehensive Benchmarking:** We conduct extensive experiments across seven representative VLA backbones and multiple manipulation environments under a unified evaluation protocol, measuring both task performance and inference efficiency. Our results provide a common empirical benchmark for understanding when training-free efficiency tricks transfer across VLA models.

II. RELATED WORK

Training-free VLA efficiency methods reduce cost along three parts of the inference loop: the visual information given to the policy, the network computation used in each policy call, and how often the policy is invoked.

Visual Efficiency. Visual computation can be reduced by allocating fidelity selectively rather than processing all image regions uniformly. Classical foveated vision preserves detail near a region of interest while compressing the periphery [17], [18], and recent robot-learning methods adopt related ideas via gaze-guided tokenization or supervision [19], [20]. However, for transformer-based VLAs with a fixed visual-token grid, pixel-space degradation alone does not necessarily reduce model computation, motivating efficiency mechanisms that act directly on visual tokens [21], [22].

VLA-Cache addresses this by reusing representations of temporally stable patches while recomputing regions that change or remain task-relevant [7]. Related work further shows that generic VLM token-pruning methods such as FastV and SparseVLM can degrade manipulation performance when transferred directly to VLAs [6], [8], [23]. This suggests that visual efficiency for VLAs should preserve

information relevant to closed-loop action generation, rather than relying solely on generic visual-language saliency.

Network Depth Reduction. Beyond reducing visual tokens, another line of work improves efficiency by reducing the amount of computation performed inside the policy network. ShortGPT ranks layers using Block Influence, while Gromov et al. remove contiguous layer groups based on representation similarity [9], [24]. Building on this idea, EfficientVLA applies non-contiguous layer pruning to VLAs without retraining [10], and MoLe-VLA learns an input-dependent layer router through self-distillation [25], [26]. Compact VLA architectures also reduce depth at design time by using fewer language-model layers or by removing the language-model pathway from action prediction [27]–[29]. Together, these methods show that some VLA backbones contain redundant depth, but the benefit of depth reduction depends strongly on the model architecture, action head, and the amount of redundancy available in the pretrained policy.

Temporal Efficiency. A complementary source of efficiency comes from temporal redundancy. In robotic manipulation, consecutive observations and actions often change slowly, which motivates frame skipping and action chunking [30]–[33]. These methods reduce the number of policy calls, but they also reduce how often the policy receives updated feedback from the environment. Therefore, recent methods use state-dependent signals to decide when computation can be safely skipped or reused. FlashVLA gates action reuse based on visual and action consistency, while SpecPrune-VLA adapts computation according to action dynamics [11], [34]. Other methods reuse temporal visual information without skipping policy calls. TTF-VLA fuses current and historical visual tokens using motion and attention relevance, VLA-InfoEntropy uses entropy-based signals, and VLA-IAP relies on interaction alignment for visual-token selection [12], [35], [36]. These approaches indicate that temporal redundancy can be exploited either at the action level or at the visual-feature level, but both require signals that are sensitive to the current interaction state.

Static and Conditional Tricks. The tricks evaluated in this work can be understood through this distinction between fixed reductions and signal-driven adaptation. Visual foveation and fixed action repetition apply the same reduction throughout execution, and therefore serve as static controls. Depth pruning reduces network computation using a signal estimated once from calibration data, following the Block Influence criterion of ShortGPT [9]. In contrast, guarded reuse follows FlashVLA and SpecPrune-VLA by checking the newest observation, recent action consistency, gripper state, and translation before deciding whether to skip a policy call [11], [34]. Temporal fusion reuses visual features across frames while still querying the policy at every step, allowing temporal visual redundancy to be studied separately from policy-call skipping [12], [35], [36].

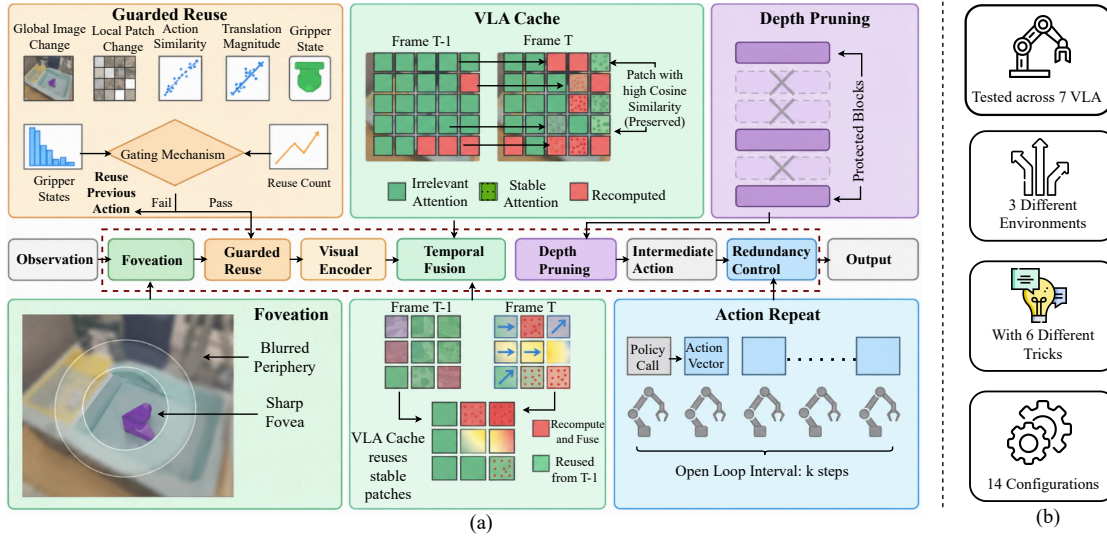


Fig. 2. (a) **Where each trick enters the control loop of a frozen VLA policy.** The center row is the inference pipeline. Foveation keeps a sharp central region and blurs the periphery. Guarded reuse gates the policy call on image change, action similarity, translation, gripper state, and reuse count. VLA-Cache and temporal fusion reuse stable patches across frames and recompute the rest. Depth pruning removes low-influence decoder blocks and protects the early and final ones. Redundancy control holds each action for k open-loop steps. (b) **Evaluation scope.** The efficiency tricks are systematically evaluated across a variety of settings for analyzing their generality and effectiveness.

III. BAG OF TRICKS

A. Overview and Design Principle

Fig. 2 summarizes the proposed efficiency mechanisms within the VLA control loop. Vision-language-action (VLA) policies inherit the flexibility of large vision-language models, but also their substantial inference cost. In robotic manipulation, slow inference limits the control frequency, while aggressive approximations can remove visual cues that are essential for contact-rich interactions. We therefore study training-free methods that reduce redundant computation while preserving closed-loop policy behavior.

At time t , a policy with fixed parameters θ maps an RGB observation I_t and language instruction ℓ to an action chunk:

$$A_t = \pi_\theta(I_t, \ell) = (a_{t,0}, \dots, a_{t,m-1}), \quad a_{t,j} \in \mathbb{R}^{d_a}, \quad (1)$$

We categorize inference redundancy along three axes: *spatial* redundancy within observations, *structural* redundancy across network blocks, and *temporal* redundancy across control steps. This perspective identifies where computation can be reduced and clarifies the failure modes introduced by each approximation. Foveation and action repeat isolate spatial and temporal perturbations, respectively, whereas depth pruning and VLA-Cache represent established structural and cache-based acceleration strategies [7], [9], [10]. Building on these approaches, we introduce two conservative mechanisms: an event-triggered action-reuse rule and an interaction-aware temporal selector. The selector shares an invariance mask between representation fusion and policy-internal caching, enabling coordinated temporal reuse while retaining task-relevant changes.

B. Foveation: Spatial Allocation of Perceptual Fidelity

Insight. This trick tests whether VLA policies require high-frequency visual detail only within a limited spatial region.

Uniformly processing all image regions is computationally expensive, although the visual cues most relevant to ma-

nipulation are often concentrated near the objects, gripper, and their interaction region. Motivated by foveated perception [17], [18], we evaluate whether pretrained VLA policies remain effective when visual fidelity decreases with distance from a central region of interest.

For a keep ratio ρ , we define a sharp central fovea with radius $r_s = \sqrt{\rho HW/\pi}$. Let $d(x)$ denote the distance from pixel x to the image center, and let $\tau(x)$ increase from zero at r_s to one at the image boundary. We construct the foveated observation as a spatially varying mixture of the original image and two Gaussian-blurred images:

$$\begin{aligned} \tilde{I}_t(x) &= w_0(x)I_t(x) + w_3(x)G_3(I_t)(x) \\ &\quad + w_9(x)G_9(I_t)(x), \end{aligned} \quad (2)$$

$$\text{s.t. } w_0(x) + w_3(x) + w_9(x) = 1.$$

Here, G_3 and G_9 are Gaussian filters with increasing blur strength, and the weights vary monotonically with $\tau(x)$. Pixels inside the fovea are preserved exactly, while pixels farther away receive progressively stronger blur. We evaluate keep ratios of 20% and 50%.

C. Action Repeat: A Temporal Redundancy Control

Insight. This trick tests how much policy inference can be reduced by exploiting short-term temporal redundancy without adapting to changes in the observed state.

Consecutive robot observations are often highly correlated, and the corresponding actions can change smoothly over short time horizons. To exploit this temporal redundancy, action repeat holds a predicted action constant across multiple control transitions. Given repeat factor k , controller applies

$$u_{t+j} = a_t, \quad j = 0, \dots, k-1, \quad (3)$$

without querying the policy during the repeated steps. For action-chunking policies, the same operation is applied to each predicted action in the chunk.

Action repeat reduces the number of policy evaluations by approximately a factor of k , but it increases the open-loop

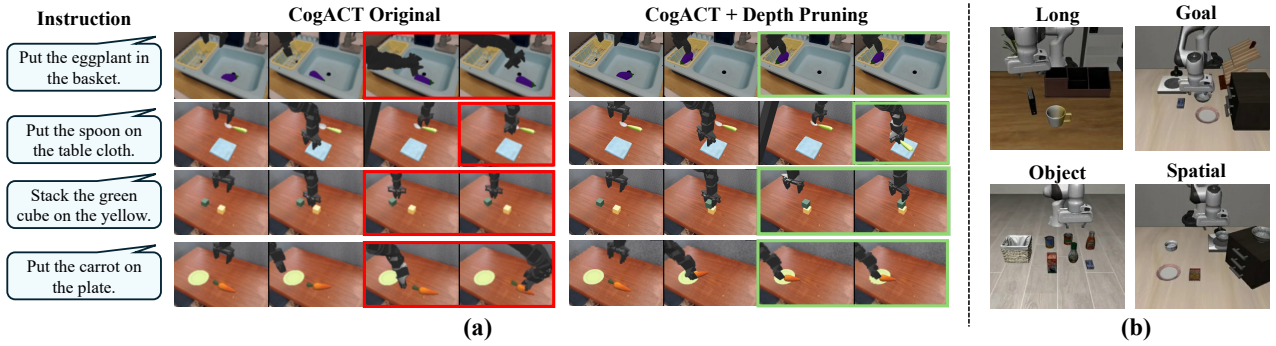


Fig. 3. **Qualitative evaluation and benchmark examples.** (a) CogACT on SimplerEnv WidowX under the original policy and with depth pruning; green/red borders denote success/failure. (b) Scenes from the four LIBERO suites.

control interval by the same factor. This can be detrimental during contact, grasp closure, and placement, where small changes in the scene may require an immediate policy update. We evaluate $k \in \{2, 4\}$ as simple temporal baselines that quantify the efficiency obtainable by disregarding state-dependent risk, thereby motivating conditional action reuse.

D. Depth Pruning: Structural Redundancy in the Decoder

Insight. This trick tests whether structurally redundant decoder blocks can be removed to reduce policy inference cost while preserving manipulation behavior.

The language-model decoder accounts for a substantial portion of OpenVLA’s inference cost. Residual architectures offer a simple criterion for identifying potentially redundant blocks: if a block induces only a small directional change in its hidden states over a calibration set, bypassing it may preserve local representations while reducing both prefill and autoregressive computation.

For decoder block j , calibration example n , and token position q , we measure block influence as

$$s_j = \frac{1}{N} \sum_{n=1}^N \left[1 - \frac{1}{Q_n} \sum_{q=1}^{Q_n} \cos(h_{j,n,q}^{\text{in}}, h_{j,n,q}^{\text{out}}) \right]. \quad (4)$$

Here, $h_{j,n,q}^{\text{in}}$ and $h_{j,n,q}^{\text{out}}$ denote the input and output hidden states of block j , respectively. Blocks with lower influence scores are treated as candidates for removal. We prune conservatively by preserving the early decoder blocks and the final block, avoiding adjacent removals, and fixing the selected blocks before evaluation. Removing these blocks reduces decoder computation while maintaining generation-cache consistency. This approximation relies on a distributional assumption: a block that appears redundant on calibration trajectories may still be important in unseen scenes, instructions, or contact states. Moreover, cosine similarity measures representational alignment rather than equivalence in the resulting action.

E. Guarded Reuse: State-Conditional Policy Evaluation

Insight. This trick tests whether policy evaluations can be skipped during temporally stable control phases while restoring closed-loop inference when state changes become salient.

Action repetition assumes that all control phases are equally predictable. Guarded reuse instead evaluates whether

the current observation supports continued execution of the previous action. The VLA policy is queried only when a perceptual or control change violates this local temporal-consistency assumption.

The guard considers global image change, maximum local-patch change, agreement between recent inferred pose actions, translation magnitude, gripper consistency, and the number of consecutive reused actions. It also rejects near-zero translational commands, for which repeated execution can cause stagnation or drift.

Let d_t^g and d_t^l denote the global and maximum local differences between inexpensive signatures of consecutive observations. Let c_t be the cosine similarity between the two most recent model-inferred six-dimensional pose actions, v_t the translation magnitude of the candidate reused action, and r_t the number of consecutive reuse steps. The decision is

$$G_t = [d_t^g \leq \tau_g] \wedge [d_t^l \leq \tau_l] \wedge [c_t \geq \tau_a] \wedge [v_t \geq \tau_v] \wedge [\text{gripper state unchanged}] \wedge [r_t < R_{\max}], \quad (5)$$

$$u_t = \begin{cases} u_{t-1}, & G_t = 1, \\ \pi_{\theta}(I_t, \ell), & G_t = 0. \end{cases}$$

The observation is checked at every control step. If any condition fails, the controller queries the original policy; $R_{\max}$ bounds the duration of open-loop execution.

F. VLA-Cache: Task-Aware Reuse of Visual Computation

Insight. This method tests whether cached visual representations can be reused for stable patches without omitting task-relevant information.

VLA-Cache [7] reduces computation within a policy evaluation by reusing cached representations for visually stable patches. To avoid reusing patches that may affect the task, it excludes patches with high language-conditioned attention.

For patch i , let $\gamma_{t,i}$ be its cosine similarity between consecutive frames and $r_{t,i}$ its text-to-vision attention score from the preceding query. The stable, task-relevant, and reusable patch sets are

$$\mathcal{S}_t = \{i : \gamma_{t,i} \geq \tau_s\}, \quad \mathcal{R}_t = \text{TopK}(r_t, K_r), \quad (6)$$

$$\mathcal{C}_t = \text{TopK}_{i \in \mathcal{S}_t}(\gamma_{t,i}, K_s) \setminus \mathcal{R}_t. \quad (7)$$

Patches in $\mathcal{C}_t$ reuse historical cache entries according to a layer-adaptive schedule. The first observation is processed

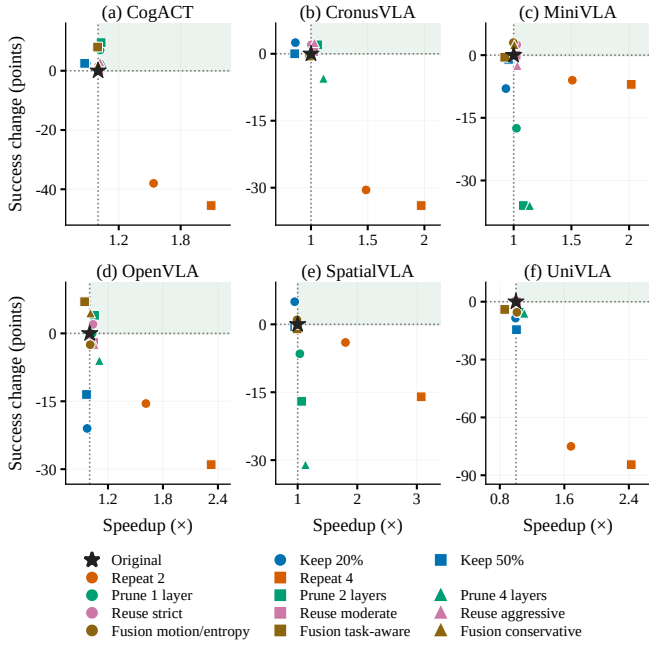


Fig. 4. **Success-speed trade-off on SimplerEnv WidowX.** Each point is one trick setting, plotted by speedup per environment step and success change relative to the original policy. Stars mark original policies, colors denote trick types, and shaded regions indicate faster, no-worse configs.

in full. We use this stability–relevance criterion to motivate the shared reuse mask in Sec. III-G.

G. Interaction-Aware Temporal Fusion

Insight. This trick tests whether temporally stable visual tokens can be reused while recomputing interaction-relevant regions.

Temporal fusion reuses projected visual tokens from temporally stable patches. Its mask protects patches that exhibit motion, contain local visual structure, receive high language-conditioned attention, or belong to externally specified interaction regions.

Let $m_{t,i}$, $e_{t,i}$, and $r_{t,i}$ denote the motion, normalized entropy, and relevance score of patch i , respectively. We form the protected set and select reusable patches as

$$\mathcal{P}_t = \text{Dilate}\left(\{i : m_{t,i} > \tau_m\} \cup \text{TopFrac}(e_t, \eta_e) \cup \text{TopFrac}(r_t, \eta_r) \cup \mathcal{P}_t^{\text{interaction}}\right), \quad (8)$$

$$\mathcal{U}_t = \text{BottomK}_{i \notin \mathcal{P}_t}(m_{t,i} + e_{t,i} + r_{t,i}, \lfloor \beta P \rfloor).$$

Let $z_{t,i}$ be the current projected token and $\tilde{z}_{t-1,i}$ the previous representation. The fused input is

$$\tilde{z}_{t,i} = \begin{cases} \tilde{z}_{t-1,i}, & i \in \mathcal{U}_t, \\ z_{t,i}, & i \notin \mathcal{U}_t. \end{cases} \quad (9)$$

The same set $\mathcal{U}_t$ is shared with the generation cache, so that stable representations are also eligible for cached computation.

IV. EXPERIMENTS AND EVALUATION

A. Experimental Setup and Protocols

Backbones. We evaluate seven VLA policies without retraining, each with its official checkpoint. CogACT [37], OpenVLA [1], SpatialVLA [2], CronusVLA [38], UniVLA [3],

and MiniVLA run on SimplerEnv, and OpenVLA, UniVLA, and SmolVLA [28] run on LIBERO.

Environments and episodes. We evaluate on SimplerEnv WidowX, Google Robot/Fractal, and LIBERO (Fig. 3). WidowX has four tasks with 50 episodes per task, while Fractal has five tasks with 50 episodes per task. LIBERO has four suites: Long, Goal, Object, and Spatial. For each backbone, all configurations use the same task instances and initial states as the original policy.

Configurations. For each backbone and environment, we evaluate the original policy and thirteen trick settings. These settings cover five training-free tricks: foveation, action repeat, depth pruning, guarded reuse, and temporal fusion. Foveation keeps 20% or 50% of the image, action repeat uses $k \in \{2, 4\}$, depth pruning removes one, two, or four layers, guarded reuse runs strict, moderate, and aggressive presets, and temporal fusion runs motion-entropy, task-aware, and conservative-adaptive settings.

Measures. We report success using the benchmark-defined success criterion. Latency is measured as wall-clock time per environment step. Avg. Steps denotes the mean episode length. Tables I and II report the best setting for each trick, chosen by highest success and then fewer average steps. The figures show all settings.

Pairing and significance. Each trick is compared with the original policy on the same episodes. We use an exact two-sided McNemar test on episodes whose outcome changes between failure and success, and treat $p < 0.05$ as significant. Since each table entry is selected from multiple settings, the corresponding p -values are used only as descriptive evidence.

Implementation details. All experiments are run on a shared GPU cluster, with hardware kept consistent within each backbone when comparing latency. All experiments are conducted using a system equipped with 4× RTX 5090 GPUs. More details on checkpoints, GPU cards, software versions, trick presets, per-cell p -values, and implementation caveats are provided at <https://anonymous.4open.science/w/icra-bot/>

B. Results and Discussion

Tables I and II report the best setting of each trick for every backbone and environment. We discuss the results through three research questions (RQs).

RQ-1: Which tricks can preserve closed-loop manipulation success across VLA backbones, and which tricks introduce new failures?

RQ-2: Which efficiency tricks provide measurable control-loop speedup in practice, and how does this speedup trade off against success rate?

RQ-3: Does the effect of an efficiency trick remain consistent across different VLA backbones and environments?

Fig. 1 shows that several efficiency tricks can preserve, and in some cases improve, manipulation success across the six WidowX backbones. The strongest positive behavior comes from guarded reuse and temporal fusion, which keep the policy close to its original closed-loop behavior while

TABLE I

SIMPLERENV RESULTS ON WIDOWX AND GOOGLE ROBOT/FRACTAL. VALUES IN PARENTHESES INDICATE THE CHANGE RELATIVE TO THE ORIGINAL METHOD OF THE SAME MODEL AND ENVIRONMENT. PARAMETER COUNTS ARE AS REPORTED BY THE ORIGINAL WORKS. UNIVLA AND MINIVLA WEIGHTS FOR GOOGLE FRACTAL ARE NOT AVAILABLE PUBLICLY.

Model	Parameters	Method	WidowX			Google Fractal		
			Success $\uparrow$ (%)	Latency $\downarrow$ (ms)	Avg. Steps $\downarrow$	Success $\uparrow$ (%)	Latency $\downarrow$ (ms)	Avg. Steps $\downarrow$
CogACT	7.6B	Original	50.00	141.41	50.85	66.80	169.06	44.78
		Foveation	52.50 (+2.50)	159.76 (+18.35)	46.57 (-4.28)	67.20 (+0.40)	194.63 (+25.57)	44.34 (-0.44)
		Action repeat	12.00 (-38.00)	92.06 (-49.35)	70.61 (+19.76)	65.60 (-1.20)	121.50 (-47.57)	46.67 (+1.89)
		Depth pruning	59.50 (+9.50)	137.48 (-3.93)	47.14 (-3.71)	66.80 (+0.00)	163.93 (-5.13)	43.92 (-0.86)
		Guarded reuse	53.00 (+3.00)	138.64 (-2.77)	49.77 (-1.08)	67.20 (+0.40)	169.94 (+0.87)	44.43 (-0.35)
		Temporal fusion	58.00 (+8.00)	141.95 (+0.54)	47.41 (-3.44)	65.60 (-1.20)	171.29 (+2.23)	44.54 (-0.23)
OpenVLA	7B	Original	43.50	210.17	50.91	35.60	239.77	64.31
		Foveation	30.00 (-13.50)	217.71 (+7.53)	62.52 (+11.61)	24.40 (-11.20)	249.43 (+9.66)	68.24 (+3.92)
		Action repeat	28.00 (-15.50)	130.26 (-79.92)	60.50 (+9.59)	40.00 (+4.40)	159.11 (-80.66)	62.91 (-1.40)
		Depth pruning	47.50 (+4.00)	199.41 (-10.76)	51.25 (+0.34)	40.80 (+5.20)	230.70 (-9.07)	61.86 (-2.46)
		Guarded reuse	45.50 (+2.00)	202.68 (-7.50)	49.69 (-1.22)	36.40 (+0.80)	236.62 (-3.15)	64.20 (-0.12)
		Temporal fusion	50.50 (+7.00)	222.24 (+12.07)	50.53 (-0.38)	42.40 (+6.80)	253.92 (+14.15)	61.44 (-2.88)
SpatialVLA	4B	Original	45.00	423.52	45.15	59.20	450.02	49.58
		Foveation	50.00 (+5.00)	445.96 (+22.43)	45.57 (+0.42)	57.20 (-2.00)	478.35 (+28.33)	49.15 (-0.43)
		Action repeat	41.00 (-4.00)	235.14 (-188.38)	49.38 (+4.23)	59.20 (+0.00)	262.88 (-187.14)	50.40 (+0.83)
		Depth pruning	38.50 (-6.50)	408.41 (-15.11)	49.70 (+4.56)	56.80 (-2.40)	426.49 (-23.53)	50.74 (+1.16)
		Guarded reuse	45.50 (+0.50)	422.60 (-0.92)	45.12 (-0.02)	60.00 (+0.80)	446.84 (-3.18)	49.44 (-0.14)
		Temporal fusion	46.00 (+1.00)	428.15 (+4.62)	45.34 (+0.19)	60.00 (+0.80)	453.38 (+3.37)	47.71 (-1.87)
CronusVLA	0.5B	Original	34.00	125.84	50.56	55.60	155.12	51.34
		Foveation	36.50 (+2.50)	145.91 (+20.07)	48.92 (-1.64)	56.80 (+1.20)	179.99 (+24.88)	50.10 (-1.24)
		Action repeat	3.50 (-30.50)	84.71 (-41.14)	73.40 (+22.84)	58.40 (+2.80)	112.37 (-42.75)	57.43 (+6.09)
		Depth pruning	36.00 (+2.00)	118.76 (-7.08)	50.52 (-0.04)	56.40 (+0.80)	152.83 (-2.29)	50.17 (-1.17)
		Guarded reuse	36.50 (+2.50)	122.13 (-3.71)	49.68 (-0.88)	56.80 (+1.20)	150.36 (-4.76)	50.73 (-0.60)
		Temporal fusion	33.50 (-0.50)	126.92 (+1.08)	51.21 (+0.65)	57.20 (+1.60)	156.90 (+1.78)	50.54 (-0.80)
UniVLA	8.5B	Original	87.50	191.00	30.21	Model weights not publicly available		
		Foveation	79.00 (-8.50)	191.81 (+0.82)	34.57 (+4.36)			
		Action repeat	12.50 (-75.00)	113.44 (-77.55)	71.93 (+41.72)			
		Depth pruning	82.00 (-5.50)	182.71 (-8.29)	31.64 (+1.43)			
		Guarded reuse	87.50 (+0.00)	189.59 (-1.41)	30.07 (-0.15)			
		Temporal fusion	87.50 (+0.00)	191.66 (+0.66)	30.21 (+0.00)			
MiniVLA	1B	Original	36.00	149.44	63.84	Model weights not publicly available		
		Foveation	35.00 (-1.00)	156.41 (+6.98)	62.91 (-0.93)			
		Action repeat	30.00 (-6.00)	99.19 (-50.24)	66.94 (+3.10)			
		Depth pruning	18.50 (-17.50)	145.92 (-3.52)	67.44 (+3.60)			
		Guarded reuse	38.50 (+2.50)	145.57 (-3.87)	62.65 (-1.19)			
		Temporal fusion	39.00 (+3.00)	149.92 (+0.49)	61.16 (-2.68)			

still allowing selective reuse. These tricks match or improve success for most backbones, and they also reduce average steps in several cases. Tables I and II show the same trend across environments, with many green cells for guarded reuse and temporal fusion.

Foveation and depth pruning also show useful cases. Foveation improves CogACT, SpatialVLA, and CronusVLA on WidowX, and gives strong gains for UniVLA on LIBERO Goal and Object. Depth pruning improves CogACT on WidowX and gives positive or matched performance for several UniVLA suites. These results show that some VLAs can tolerate reduced visual fidelity or reduced decoder depth, suggesting that pretrained policies contain exploitable slack. At the same time, the benefit depends on the trick and the backbone. Action repeat gives large speed benefits but is less suitable as a success-preserving method, especially when repeated actions extend the open-loop interval. Foveation and depth pruning are also more model-dependent than guarded reuse and temporal fusion.

[A-1] Conservative tricks such as guarded reuse and temporal fusion are the safest for preserving success. Foveation and depth pruning can also improve selected backbones, while action repeat mainly serves as a speed-oriented baseline.

of efficiency. Action repeat gives the largest control-loop speedup because it directly reduces the number of policy calls. This makes it an upper-bound baseline for temporal reuse. Depth pruning gives a more model-internal speedup by reducing decoder computation (Fig. 5). In several cases, it lowers latency while keeping success close to, or above, the original policy, showing that some decoder layers can be bypassed without breaking closed-loop behavior.

Guarded reuse provides a more conservative speed–success trade-off. Instead of skipping policy calls at fixed intervals, it reuses actions only when visual change, action agreement, and gripper state remain stable. As a result, it gives smaller but safer speedups, while preserving success across many backbones. This shows that event-triggered policy evaluation is a practical way to exploit temporal redundancy without fully committing to open-loop execution.

Temporal fusion and foveation mainly improve the representation side rather than directly reducing policy FLOPs. Their benefit is therefore not raw speed, but stability: they can preserve or improve success while changing what information is emphasized.

[A-2] Action repeat gives the largest speedup, depth pruning gives practical decoder-level savings, and guarded reuse gives the safest speed–success trade-off.

Fig. 4 shows that different tricks provide different forms

However, the effect of an efficiency trick is not consistent

TABLE II

LIBERO RESULTS ACROSS FOUR TASK SUITES. VALUES IN PARENTHESES INDICATE THE CHANGE RELATIVE TO THE ORIGINAL METHOD WITHIN THE SAME TASK SUITE. PARAMETER COUNTS ARE AS REPORTED BY THE ORIGINAL WORKS.

Task	Method	UniVLA (8.5B)			OpenVLA (7B)			SmoIVLA (0.45B)		
		Success $\uparrow$ (%)	Latency $\downarrow$ (ms)	Avg. Steps $\downarrow$	Success $\uparrow$ (%)	Latency $\downarrow$ (ms)	Avg. Steps $\downarrow$	Success $\uparrow$ (%)	Latency $\downarrow$ (ms)	Avg. Steps $\downarrow$
Long	Original	86.00	59.85	280.36	56.00	171.55	396.79	42.00	289.48	428.35
	Foveation	90.00 (+4.00)	60.58 (+0.73)	268.88 (-11.48)	44.00 (-12.00)	178.55 (+7.00)	410.31 (+13.52)	21.00 (-21.00)	306.48 (+17.00)	477.84 (+49.49)
	Action repeat	27.00 (-59.00)	35.90 (-23.95)	465.50 (+185.14)	46.00 (-10.00)	93.16 (-78.39)	407.47 (+10.68)	31.00 (-11.00)	152.13 (-137.36)	443.45 (+15.10)
	Depth pruning	89.00 (+3.00)	57.22 (-2.63)	282.61 (+2.25)	53.00 (-3.00)	168.38 (-3.17)	398.62 (+1.83)	9.00 (-33.00)	291.58 (+2.10)	500.07 (+71.72)
	Guarded reuse	86.00 (+0.00)	59.48 (-0.37)	280.58 (+0.22)	51.00 (-5.00)	163.76 (-7.79)	396.62 (-0.17)	42.00 (+0.00)	268.93 (-20.55)	426.73 (-1.62)
	Temporal fusion	87.00 (+1.00)	64.88 (+5.03)	281.91 (+1.55)	49.00 (-7.00)	172.91 (+1.35)	410.34 (+13.55)	45.00 (+3.00)	276.74 (-12.74)	415.94 (-12.41)
Goal	Original	92.00	65.10	122.58	75.00	176.09	157.59	80.00	295.68	147.66
	Foveation	99.00 (+7.00)	65.49 (+0.39)	114.37 (-8.21)	75.00 (+0.00)	181.32 (+5.23)	157.29 (-0.30)	67.00 (-13.00)	304.08 (+8.40)	168.51 (+20.85)
	Action repeat	34.00 (-58.00)	39.75 (-25.35)	244.05 (+121.47)	68.00 (-7.00)	96.25 (-79.84)	175.16 (+17.57)	72.00 (-8.00)	154.18 (-141.50)	164.22 (+16.56)
	Depth pruning	93.00 (+1.00)	63.19 (-1.91)	121.69 (-0.89)	72.00 (-3.00)	170.53 (-5.56)	165.43 (+7.84)	62.00 (-18.00)	293.54 (-2.14)	187.03 (+39.37)
	Guarded reuse	93.00 (+1.00)	64.69 (-0.41)	121.34 (-1.24)	72.00 (-3.00)	157.86 (-18.23)	163.88 (+6.29)	79.00 (-1.00)	270.43 (-25.25)	148.58 (+0.92)
	Temporal fusion	95.00 (+3.00)	64.96 (-0.14)	122.07 (-0.51)	77.00 (+2.00)	176.23 (+0.14)	158.66 (+1.07)	78.00 (-2.00)	279.27 (-16.41)	150.04 (+2.38)
Object	Original	96.00	62.41	145.82	87.00	173.85	165.89	94.00	290.85	162.25
	Foveation	100.00 (+4.00)	63.00 (+0.59)	136.04 (-9.78)	86.00 (-1.00)	180.11 (+6.26)	166.18 (+0.29)	81.00 (-13.00)	309.09 (+18.24)	176.39 (+14.14)
	Action repeat	21.00 (-75.00)	37.61 (-24.79)	266.12 (+120.30)	84.00 (-3.00)	93.76 (-80.09)	173.96 (+8.07)	90.00 (-4.00)	150.16 (-140.69)	169.02 (+6.77)
	Depth pruning	99.00 (+3.00)	61.65 (-0.76)	143.23 (-2.59)	85.00 (-2.00)	168.38 (-5.47)	170.03 (+4.14)	73.00 (-21.00)	294.80 (+3.95)	193.35 (+31.10)
	Guarded reuse	96.00 (+0.00)	62.68 (+0.27)	145.82 (+0.00)	85.00 (-2.00)	169.28 (-4.57)	165.41 (-0.48)	90.00 (-4.00)	272.67 (-18.18)	163.60 (+1.35)
	Temporal fusion	98.00 (+2.00)	61.38 (-1.03)	142.56 (-3.26)	86.00 (-1.00)	185.64 (+11.79)	165.91 (+0.02)	90.00 (-4.00)	278.99 (-11.86)	162.20 (-0.05)
Spatial	Original	95.00	72.29	108.31	78.00	181.02	133.91	76.00	292.28	132.92
	Foveation	96.00 (+1.00)	73.90 (+1.61)	108.93 (+0.62)	83.00 (+5.00)	187.09 (+6.08)	129.40 (-4.51)	76.00 (+0.00)	313.86 (+21.58)	135.92 (+3.00)
	Action repeat	27.00 (-68.00)	46.40 (-25.89)	208.63 (+100.32)	72.00 (-6.00)	101.16 (-79.86)	142.05 (+8.14)	64.00 (-12.00)	157.71 (-134.57)	148.18 (+15.26)
	Depth pruning	96.00 (+1.00)	71.59 (-0.70)	107.83 (-0.48)	62.00 (-16.00)	175.02 (-5.99)	149.98 (+16.07)	62.00 (-14.00)	296.09 (+3.81)	151.44 (+18.52)
	Guarded reuse	95.00 (+0.00)	73.31 (+1.02)	108.31 (+0.00)	82.00 (+4.00)	177.48 (-3.53)	129.59 (-4.32)	78.00 (+2.00)	282.15 (-10.13)	130.84 (-2.08)
	Temporal fusion	95.00 (+0.00)	72.20 (-0.09)	108.31 (+0.00)	85.00 (+7.00)	180.80 (-0.22)	125.72 (-8.19)	82.00 (+6.00)	285.17 (-7.11)	126.14 (-6.78)

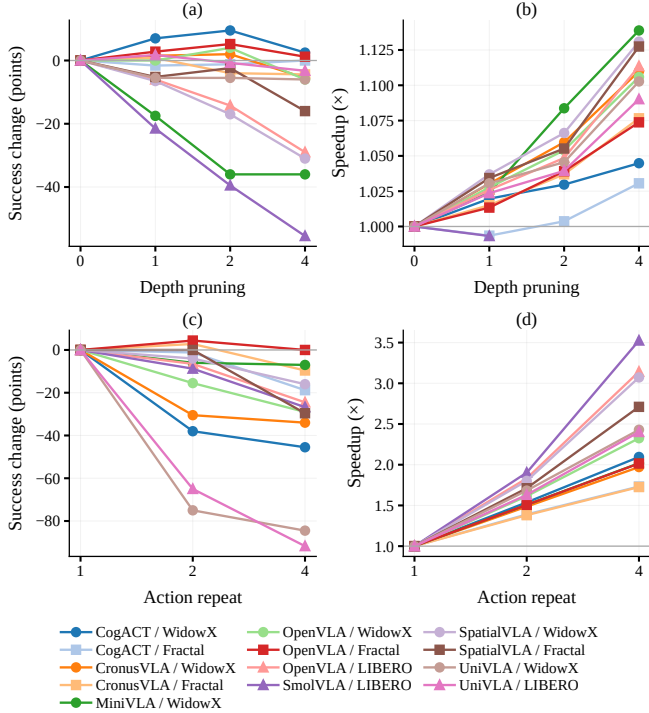


Fig. 5. Depth pruning (a, b) and action repeat (c, d) on every backbone and environment, success change (a, c) and speedup (b, d) against the number of removed decoder layers or the number of times each action is repeated. Speedup is the original’s time per step divided by the trick’s, and LIBERO suites are pooled per backbone, so 13 lines stand for the 22 pairs.

across VLA backbones and environments. The same trick can help one model, leave another unchanged, and hurt a third. For example, depth pruning improves CogACT on WidowX, but reduces success for MiniVLA and SpatialVLA on the same benchmark. This shows that structural redundancy is not a fixed property of the trick alone; it also depends on how the backbone uses its decoder layers during control.

A similar pattern appears for visual changes. Foveation improves UniVLA on several LIBERO suites, but reduces SmoIVLA on the same suites. Tolerance to non-uniform

visual fidelity is thus backbone-specific. Action repeat also depends on the environment: it is often harmful on WidowX, where short-horizon contact and correction matter, but is less damaging on Fractal for some backbones.

[A-3] Efficiency tricks therefore do not transfer as plug-and-play improvements. Their behavior is a joint property of the trick, the backbone, and the task distribution, so each trick needs matched evaluation under the target backbone and environment.

Additional details on evaluation setup, methodology, key assumptions, and limitations are provided on the project page <https://anonymous.4open.science/w/cra-bot/>

Additional qualitative results and illustrative demonstrations are included in the accompanying video.

V. CONCLUSION

In this work, we studied six policy variants, including the original policy and five efficiency tricks, across seven open-source VLA policies and three manipulation environments. Our results showed that different efficiency tricks exposed different operating regimes. Guarded reuse was the most reliable, preserving success while providing modest speedups when its gates opened. Action repetition provided the largest speedup, but also showed the risk of long open-loop execution. Depth pruning provided decoder-level savings, but its benefit depended on the backbone. Foveation and temporal fusion tested how much the visual representation could change while keeping the policy stable. Our main finding was that VLA efficiency was policy- and environment-dependent. A trick was useful not only when it improved success, but also when it revealed which parts of inference could be safely reused, removed, or modified. This made matched evaluation important before applying any efficiency trick to a new VLA. We hope that this research will help future VLA efficiency methods move beyond isolated speedups toward reliable, matched, and deployment-aware evaluation.

REFERENCES

- [1] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024. 1, 5
- [2] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li, “SpatialVLA: Exploring spatial representations for visual-language-action model,” *arXiv preprint arXiv:2501.15830*, 2025. 1, 5
- [3] Y. Wang, X. Li, W. Wang, J. Zhang, Y. Li, Y. Chen, X. Wang, and Z. Zhang, “Unified vision-language-action model,” *arXiv preprint arXiv:2506.19850*, 2025. 1, 5
- [4] W. Guan, Q. Hu, A. Li, and J. Cheng, “Efficient vision-language-action models for embodied manipulation: A systematic survey,” *arXiv preprint arXiv:2510.17111*, 2025. 1
- [5] Z. Yu, B. Wang, P. Zeng, H. Zhang, J. Zhang, Z. Wang, L. Gao, J. Song, N. Sebe, and H. T. Shen, “A survey on efficient vision-language-action models,” *arXiv preprint arXiv:2510.24795*, 2026. 1
- [6] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, “An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models,” *arXiv preprint arXiv:2403.06764*, 2024. 1, 2
- [7] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, “VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching,” *arXiv preprint arXiv:2502.02175*, 2025. 1, 2, 3, 4
- [8] Z. Liu, Y. Chen, H. Cai, T. Lin, S. Yang, Z. Liu, and B. Zhao, “VLA-Pruner: Temporal-aware dual-level visual token pruning for efficient vision-language-action inference,” *arXiv preprint arXiv:2511.16449*, 2025. 1, 2
- [9] X. Men, M. Xu, Q. Zhang, B. Wang, H. Lin, Y. Lu, X. Han, and W. Chen, “ShortGPT: Layers in large language models are more redundant than you expect,” *arXiv preprint arXiv:2403.03853*, 2024. 1, 2, 3
- [10] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, “EfficientVLA: Training-free acceleration and compression for vision-language-action models,” *arXiv preprint arXiv:2506.10100*, 2025. 1, 2, 3
- [11] X. Tan, Y. Yang, P. Ye, J. Zheng, B. Bai, X. Wang, J. Hao, and T. Chen, “Think twice, act once: Token-aware compression and action reuse for efficient inference in vision-language-action models,” *arXiv preprint arXiv:2505.21200*, 2025. 1, 2
- [12] C. Liu, J. Zhang, C. Li, Z. Zhou, S. Wu, S. Huang, and H. Duan, “TTF-VLA: Temporal token fusion via pixel-attention integration for vision-language-action models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. arXiv:2508.19257. 1, 2
- [13] S. Choi, Y. Lee, Y. Park, C. D. Kim, R. Krishna, D. Fox, and Y. Yu, “vla-eval: A unified evaluation harness for vision-language-action models,” *arXiv preprint arXiv:2603.13966*, 2026. 1
- [14] StarVLA Community and Von Neumann Institute, HKUST, “StarVLA: A lego-like codebase for vision-language-action model developing,” *arXiv preprint arXiv:2604.05014*, 2026. 1
- [15] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, “Bag of tricks for image classification with convolutional neural networks,” in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 558–567, IEEE, 2019. 1
- [16] F. Liu, W. Chao, N. Tan, and H. Liu, “Bag of tricks for inference-time computation of llm reasoning, 2025,” URL <https://arxiv.org/abs/2502.07191>. 1
- [17] E. L. Schwartz, “Computational anatomy and functional architecture of striate cortex: A spatial mapping approach to perceptual coding,” *Vision Research*, vol. 20, no. 8, pp. 645–669, 1980. 2, 3
- [18] V. J. Traver and A. Bernardino, “A review of log-polar imaging for visual perception in robotics,” *Robotics and Autonomous Systems*, vol. 58, no. 4, pp. 378–398, 2010. 2, 3
- [19] I. Chuang, J. Zou, A. Lee, D. Gao, and I. Soltani, “Look, focus, act: Efficient and robust robot learning via human gaze and foveated vision transformers,” *arXiv preprint arXiv:2507.15833*, 2026. 2
- [20] A. Pani and Y. Yang, “Gaze-regularized vision-language-action models for robotic manipulation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 11332–11342, 2026. 2
- [21] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman, “Token merging: Your ViT but faster,” in *International Conference on Learning Representations (ICLR)*, 2023. 2
- [22] S. Debnath, B. D. Manh, Z. Liu, and L. Wang, “Llmind: Bio-inspired training-free adaptive visual representations for vision-language models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3133–3142, 2026. 2
- [23] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng, T. Huang, K. Cheng, D. Gudovskiy, T. Okuno, Y. Nakata, K. Keutzer, and S. Zhang, “SparseVLM: Visual token sparsification for efficient vision-language model inference,” *arXiv preprint arXiv:2410.04417*, 2025. 2
- [24] A. Gromov, K. Tirumala, H. Shapourian, P. Gloriosio, and D. A. Roberts, “The unreasonable ineffectiveness of the deeper layers,” *arXiv preprint arXiv:2403.17887*, 2025. 2
- [25] R. Zhang, M. Dong, Y. Zhang, L. Heng, X. Chi, G. Dai, L. Du, D. Wang, Y. Du, and S. Zhang, “MoLe-VLA: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation,” *arXiv preprint arXiv:2503.20384*, 2025. 2
- [26] Y. Yue, Y. Wang, B. Kang, Y. Han, S. Wang, S. Song, J. Feng, and G. Huang, “DeeR-VLA: Dynamic inference of multimodal large language models for efficient robot execution,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2411.02359. 2
- [27] M. Reuss, H. Zhou, M. Rühle, Ö. E. Yağmurlu, F. Otto, and R. Lioutikov, “FLOWER: Democratizing generalist robot policies with efficient vision-language-action flow policies,” *arXiv preprint arXiv:2509.04996*, 2025. 2
- [28] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025. 2, 5
- [29] H. Xie, C. Yao, X. Wu, Y. Zhu, D. Liang, X. Bai, and H. Ding, “TurboVLA: Real-time vision-language-action model at 32 Hz on an RTX 4090 with <1 GB VRAM,” *arXiv preprint arXiv:2607.27205*, 2026. 2
- [30] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. 2
- [31] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS)*, 2023. 2
- [32] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023. 2
- [33] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025. 2
- [34] H. Wang, J. Xu, Y. Xiang, J. Pan, Y. Zhou, Y.-L. Li, and G. Dai, “SpecPrune-VLA: Accelerating vision-language-action models via action-aware self-speculative pruning,” *arXiv preprint arXiv:2509.05614*, 2026. 2
- [35] C. Liu, Y. He, Z. Kang, X. Qu, and J. Wang, “VLA-InfoEntropy: A training-free vision-attention information entropy approach for vision-language-action models inference acceleration and success,” *arXiv preprint arXiv:2604.05323*, 2026. 2
- [36] J. Cheng, H. Wang, W. Li, G. Wang, Y. Zhang, X. Tang, J. Wu, X. Chen, Y. Liu, and W. Zhang, “VLA-IAP: Training-free visual token pruning via interaction alignment for vision-language-action models,” *arXiv preprint arXiv:2603.22991*, 2026. 2
- [37] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, *et al.*, “Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation,” *arXiv preprint arXiv:2411.19650*, 2024. 5
- [38] H. Li, S. Yang, Y. Chen, X. Chen, X. Yang, Y. Tian, H. Wang, T. Wang, D. Lin, F. Zhao, *et al.*, “Cronusvla: Towards efficient and robust manipulation via multi-frame vision-language-action modeling,” *arXiv preprint arXiv:2506.19816*, 2025. 5